

ROS — Solution Design Document

Generated: 2026-06-17 10:23

1. Executive Summary

This Solution Design Document describes the architecture, schema, and data-generation approach for the ROS (Restaurant Operations System) dataset. The dataset comprises 15 normalised tables — 9 master data tables and 6 transactional tables — covering one full trading year (365 days) of restaurant operations across 50 restaurants in the UK and India. All foreign-key relationships, financial formulas, and business rules specified in the milestone documents have been implemented and verified by 79 automated validation checks. The Deliveries table contains a deliberate 5% (10,950 rows) of match_status=FALSE entries — real reconciliation anomalies that the match_status flag is designed to surface for downstream investigation.

2. Schema Overview

The schema follows a normalised relational design. Each entity from the ROS dataset becomes a discrete table, linked by primary and foreign keys. The tables are listed below in dependency order — master data first, then transactional data.

#	Table	Type	Rows	Columns	Primary Key
1	Countries	master	249	5	country_alpha2_code
2	Currencies	master	4	3	currency_id
3	TaxInfo	master	2	4	tax_type_id
4	Departments	master	9	3	Department_id
5	Roles	master	4	5	id
6	Subscriptions	master	4	10	subscription_id
7	Clients	master	30	10	client_id
8	Restaurants	master	50	10	restaurant_id
9	Users	master	300	13	user_id
10	Orders	transactional	547,500	13	order_id
11	Sales	transactional	18,250	9	sales_id
12	Expenses	transactional	18,250	8	expense_id
13	Deliveries	transactional	219,000	7	delivery_id
14	Cash_Up	transactional	18,250	12	cashup_id
15	Banking	transactional	18,250	8	banking_id

Total rows across all tables: 840,152

3. Master Data

Countries (249 rows) — ISO 3166-1 country reference list, preserved from the source dataset. Used for geographic classification of clients and restaurants.

Currencies (4 rows) — GBP, EUR, USD, INR with currency symbols.

TaxInfo (2 rows) — Tax rates by country: UK VAT 8%, India GST 18%. The restaurant's country determines which rate applies to its orders.

Departments (9 rows) — 9 departments: Payroll, Human Resources, Accounting, Quality Assurance, Advertising, Finances, Sales and Marketing, Legal Department, Customer Service.

Roles (4 rows) — 4 roles with sequential IDs 1-4: Front Office Staff, Chef, Supervisor, Admin. The source Roles table had IDs 1, 2, 3, 5 (skipping 4); the regenerated table has sequential IDs to eliminate orphan references in Users.role_id.

Subscriptions (4 rows) — 4 tiers with per-client user limits: Free (10 users, £0/mo), Freemium (25 users, £49/mo), Premium (50 users, £199/mo), Enterprise (200 users, £499/mo). User counts per client respect these limits per Project Requirements Docket Rule #2.

Clients (30 rows) — 30 client companies: 20 UK + 10 India (67/33 split), 3 inactive (10%). Each client carries an activation_date and (if inactive) an inactivation_date strictly after the activation_date.

Restaurants (50 rows) — 50 restaurants: 40 UK + 10 India (80/20 split). Each restaurant belongs to a client and inherits the client's currency and (by extension) country for tax purposes.

Users (300 rows) — 300 users, 30 inactive (10%). Average 10 users per client, 6 users per restaurant. Each user has a valid email (no spaces, well-formed), valid region-specific phone, and references valid role, department, restaurant, client, and subscription.

4. Transactional Data

Orders (547,500 rows) — Generated as 50 restaurants × 30 orders/day × 365 days. 60% Dine-In (service charge 5% of base), 40% Home Delivery (delivery charge 10% of base). The 40/60 split is enforced deterministically to produce exactly 219,000 deliveries. Order amounts, tax (8% UK / 18% India), and totals are computed by vectorised numpy operations.

Sales (18,250 rows) — One record per restaurant per day (50 × 365). Aggregated from Orders using numpy bincount by (restaurant_id × day_idx). Includes a random credit card tip between 0 and 10 per restaurant-day.

Expenses (18,250 rows) — One record per restaurant per day. Components: bills (mean £150, capped <£1,000), vendors (mean £40, capped <£200), wage advances (mean £200, capped <£1,000), repairs (mostly <£1,000 with 5% chance of £1,000-£1,500), sundries (£1-£100). Exponential distribution keeps daily expenses below daily sales on most days, ensuring EOD stays positive.

Deliveries (219,000 rows) — One record per Home Delivery order (40% of Orders). Partner randomly assigned between UberEATS and JustEats. $\text{api_amount} = (\text{food} + \text{drinks} + \text{delivery_charges}) \times (1 - 6\%)$ per the partner-share formula. `match_status` indicates whether `order_amount` equals `api_amount` (the reconciliation audit flag). Column is named `match_status` (not 'match') per the schema fix. Distribution is enforced at 95% TRUE (208,050 rows) / 5% FALSE (10,950 rows) — the 5% mismatches are real reconciliation anomalies traceable by `delivery_id`, with a discrepancy between £0.50 and £5.00. This matches the Milestone-Three framing: 'Any discrepancy is flagged as a mismatch requiring investigation.'

Cash_Up (18,250 rows) — End-of-day reconciliation per restaurant per day. BOD Day 1 = £0; subsequent days' BOD = previous EOD – previous `banked_total`. $\text{EOD} = \text{BOD} + \text{sales} - \text{expenses} - \text{delivery_charges}$ (sales excludes tax). The carry-over chain is processed in strict date order per restaurant to avoid the cascade errors noted in Milestone-Eight.

Banking (18,250 rows) — Bank deposit records paired 1:1 with `Cash_Up`. $\text{banking_total} = \text{EOD}$ from `Cash_Up`; $\text{banked_total} = \text{random amount between } \pounds 10 \text{ and EOD (or 0 if EOD} \leq 0)$. `reconcile_status` flags Reconciled (`banked = EOD`), Partial ($0 < \text{banked} < \text{EOD}$), or Unreconciled (`banked = 0`). `sealed_by` references a staff name from the restaurant's user pool.

5. Business Rules & Formulas

Order composition — Every order has three monetary components (drinks, food, other). Dine-In orders add a 5% service charge on the base. Home Delivery orders add a 10% delivery charge on the base. $\text{order_amount} = \text{drinks} + \text{food} + \text{other} + \text{service_charges} + \text{delivery_charges}$. $\text{Tax} = \text{order_amount} \times \text{rate}$ (8% UK, 18% India). $\text{order_total} = \text{order_amount} + \text{tax}$.

Delivery partner split — The 10% delivery charge is split: 6% to the partner (UberEATS / JustEats), 4% retained by the restaurant. $\text{api_amount} = (\text{food} + \text{drinks} + \text{delivery_charges}) \times (1 - 6\%)$. The `match_status` flag in the Deliveries table enables reconciliation audits.

Cash-Up chain — Each restaurant performs a cash-up at the end of every trading day. BOD Day 1 = £0; subsequent BOD = previous EOD – previous `banked_total`. $\text{EOD} = \text{BOD} + \text{sales} - \text{expenses} - \text{delivery_charges}$ (sales excludes tax). The chain is processed in strict date order.

Banking — A random amount between £10 and the EOD is deposited; the remainder carries forward to the next day's BOD. This implements the partial banking practice described in Milestone-Eight.

Subscription limits — Per Project Requirements Docket Rule #2, user counts per client respect the subscription tier limits (Free=10, Freemium=25, Premium=50, Enterprise=200). Total 300 users across 30 clients (10 per client on average) sits well within tier capacity.

Active/Inactive logic — 10% of clients and 10% of users are inactive. Active records have NULL inactivation_date. Inactive records have an inactivation_date strictly after the activation_date.

Tax jurisdiction — Tax follows the restaurant's country, NOT the client's country. This is verified by 547,500 individual checks in the validator.

6. Validation

The dataset is verified by 79 automated validation checks across 6 categories:

- **Schema validation** — column names, order, and count match spec for all 15 tables.
- **Referential integrity** — every FK references an existing PK (16 FK relationships checked).
- **Primary key uniqueness** — no nulls, no duplicates across all 15 PKs.
- **Domain validation** — email/phone format, status enums, country codes.
- **Arithmetic validation** — order formulas (547,500 orders), sales roll-up (18,250 rows), delivery api_amount + match_status consistency (219,000 rows), delivery 95/5 match distribution, cash-up/banking chain (18,250 rows × 365-day carry-over).
- **Business-rule validation** — subscription tier limits, role assignments, active/inactive ratios, geographic distribution, tax jurisdiction, date range.

All 79 checks pass. See Validation_Report.txt for the full checklist.

7. File Inventory

File	Description
ROS_Dataset.xlsx	Main dataset — 15 sheets, 827,832 total rows, 48.6 MB
ROS-Data Dictionary.xlsx	Data dictionary — every table, column, datatype, rule
Data Generation Strategy - ROS.xlsx	Generation strategy — method, count, rules per dataset
Data Transformation Tracker - ROS.xlsx	Transformation tracker — 30 transformations applied
Data Cleaning Matrix.xlsx	Cleaning matrix — 15 datasets tracked
Data Anomalies- ROS.xlsx	Anomaly sheet — 1 real anomaly category: 10,950 delivery match_status=FALSE
Solution Design Document.pdf	This document
Validation_Report.txt	Full validation output (79 checks pass)